AI Guard — Interview Preparation Guide

Capgemini Exceller 2027 — technical interview

Project: `eslint-plugin-ai-guard` (AI Guard) — ESLint plugin + CLI + GitHub Action + MCP server + Claude Code hooks

> **How this guide was built.** Every statement comes from files in this repository (source, tests, `package.json`, `action.yml`, `docs/`, `CHANGELOG.md`, the older `AI-GUARD-TECHNICAL-INTERVIEW-GUIDE.md` and `PROJECT-DEEP-DIVE.md`). No tests, builds, lint runs, scans or web searches were performed while writing it (only file reading; the PDF conversion at the end used tools already on this machine). Where a claim could not be confirmed from the repository it is marked **Not verified from repository**. Where the older guides disagree with the current code, the code wins and the difference is listed in Section 0.

Table of contents

0. Read this first: corrections to older documents
1. Project overview
2. High-Level Design (HLD)
3. Low-Level Design (LLD)
4. AST explained with examples
5. Technologies and concepts
6. Expected technical questions and answers
7. Code walkthrough
8. System design questions
9. AI-assisted development questions
10. Final rapid revision

---

0. Read this first: corrections to older documents

The repository has older write-ups (`AI-GUARD-TECHNICAL-INTERVIEW-GUIDE.md`, `PROJECT-DEEP-DIVE.md`, `docs/architecture.md`). Some numbers and claims in them are out of date. **Use the numbers below** in the interview.

Topic	Older document says	Current repository shows	Evidence
Version	`1.4.0`	`1.4.3` in `package.json` (changelog marks 1.4.3 as *unreleased*; 1.4.2 is the last released entry)	`package.json:3`, `CHANGELOG.md`

Topic	Older document says	Current repository shows	Evidence
CLI commands	11 (or 9 in the deep dive)	**12** registered in `cli/index.ts`: `run`, `changed`, `integrity`, `init`, `doctor`, `preset`, `ignore`, `baseline`, `init-context`, `report`, `init-claude`, `claude-hook`	`cli/index.ts`
MCP tools	3	**5** tools (`ai_guard_scan_file`, `ai_guard_scan_diff`, `ai_guard_scan_project`, `ai_guard_check_integrity`, `ai_guard_rules`) + 1 prompt (`verify_changes`)	`src/mcp/create-server.ts`
Test count	829 tests / 44 files (guide), 667 / 39 (`CLAUDE.md`, `architecture.md`), 678 / 39 (deep dive)	1,042 tests in 57 files, as recorded in the release audit	`docs/release/1.4.3-readiness-audit.md` (**I did not re-run tests for this guide**)
Secret rule "entropy heuristic"	"entropy and whitelist heuristic"	**No entropy calculation exists in the code.** It uses: a name regex, a placeholder list, a minimum length of 8, and a hash/digest suffix filter	`src/rules/security/no-hardcoded-secret.ts`
SARIF fingerprint	"hash of file + ruleId + message … stable when line numbers shift"	The hash input is `ruleId : uri(lowercase) : startLine : message`. **The line number IS included**, so a moved finding gets a new fingerprint	`cli/utils/sarif.ts` → `generateStableFingerprint`
`docs/architecture.md` file list	mentions `cli/utils/github-env.ts` and `cli/utils/fail-on.ts`	These files do not exist in `cli/utils/`. `isGitHubActions()` lives in `git-diff.ts`; exit-code logic (`resolveExitCode`) lives in `cli/commands/run.ts`	`cli/utils/`, `cli/commands/run.ts`
Autofix	4 autofix rules, 3 with suggestions	**Verified correct**: `fixable: 'code'` in `no-await-in-loop`, `no-floating-promise`, `no-empty-catch`, `no-hardcoded-secret`; `hasSuggestions` in `no-console-in-handler`, `no-async-without-await`, `no-empty-catch`	`grep` of `src/rules/`
Presets	"3 presets" in some docs	**4**: `recommended`, `strict`, `security`, `agent`	`src/index.ts`, `src/configs/`

1. Project overview

1.1 What AI Guard is

AI Guard (`eslint-plugin-ai-guard`) is a **deterministic static-analysis tool for JavaScript and TypeScript** that finds code patterns AI coding assistants commonly produce and that cause bugs or security problems. It ships as:

Piece	What it is	Where
ESLint plugin	18 custom rules + 4 presets	`src/`
CLI (`ai-guard`)	Zero-config scanner, changed-file scanner, baseline, HTML report, setup helpers	`cli/`
GitHub Action	Runs the CLI on pull requests, writes SARIF/annotations/summary	`action.yml` , `action/entrypoint.js`
MCP server (`ai-guard-mcp`)	Lets an AI agent call the scanner as tools over STDIO	`src/mcp/`
Claude Code hooks	`ai-guard claude-hook` feeds findings back to Claude after edits	`cli/commands/claude-hook.ts` , `cli/commands/init-claude.ts`
Check integrity (new in 1.4.3)	Detects changes that *weaken* checks: new `eslint-disable` , skipped tests, lowered coverage, removed CI steps	`cli/utils/integrity/`

The README states the positioning: "*Deterministic AST safety layer and CI guardrails for AI-assisted JavaScript and TypeScript code.*" and explicitly says what it is **not**: it is **not an AI detector** (it does not guess who wrote the code) and **does not replace ESLint**.

1.2 The problem it solves

From the README's "Why AI Guard?" table and `PROJECT-DEEP-DIVE.md` : AI assistants write code quickly, but repeatedly produce a small set of dangerous patterns that ordinary linters miss:

Pattern	Why AI produces it (README)	Impact
Floating promises	Omits `await` / `.catch()`	Silent failures, unhandled rejections
`array.map(async …)`	Passes async callback to array methods	Returns `Promise[]` , not values
`await` in loops	Sequential awaits	Latency
Empty `catch {}`	Generates generic try/catch	Swallowed errors
Hardcoded secrets	Inserts placeholder/real keys	Credential leakage
`eval()`	Dynamic code compilation	Code injection
SQL string concatenation	Builds queries from variables	SQL injection
`JSON.parse(req.body)`	No validation	Crashes / DoS
Routes without auth	Boilerplate endpoints	Unprotected APIs, IDOR
`if (true)` dead branches	Scaffolding from prompt iterations	Dead code

1.3 Why it was developed (what the repository says)

- **AI code looks right but isn't.** `PROJECT-DEEP-DIVE.md` §2 and the older guide §6 argue that LLMs optimise for plausible syntax with a limited context window, and human reviewers trust clean-looking code.
- **Existing tools are heavy or not AI-aware.** `docs/benchmarks.md` compares AI Guard with ESLint core and `@typescript-eslint`: 17 of 18 rules need **no type information / no `tsconfig.json`**, which is why it runs fast and zero-config.
- **Adoption-first design.** `CONTRIBUTING.md` lists the philosophy: *precision over recall*, low false positives, `recommended` must be safe on day one, AI-specific focus, GitHub-native output, and self-validation (AI Guard scans its own source with the strict preset).

> Why the founder personally wanted it (motivation story): **Not verified from repository.** Prepare your own 2 sentences (see Section 9).

1.4 Technology stack (verified from `package.json`, `tsup.config.ts`, `tsconfig*.json`)

Layer	Technology
Language	TypeScript (strict mode), compiled to ES2022; runs on Node.js **`>= 20`** (`engines`)
Rule authoring	`@typescript-eslint/utils` (`ESLintUtils.RuleCreator`, `AST_NODE_TYPES`, `TSESTree`) — the **only runtime dependency**
Host	ESLint (peer dependency, `>=8.0.0`), optional peer `@typescript-eslint/parser`
CLI	`commander`, `chalk`, `ora`, `@inquirer/prompts` (bundled at build time; devDependencies)
MCP	`@modelcontextprotocol/sdk`, `zod` (bundled)
YAML parsing (integrity)	`js-yaml` (bundled devDependency)
Build	`tsup` → 3 bundles: plugin (CJS + ESM + `.d.ts`), CLI (CJS), MCP server (CJS)
Tests	`vitest`, `@typescript-eslint/rule-tester`
Docs	`vitepress`
CI	GitHub Actions (`ci.yml`, `ai-guard.yml`, `integrity-e2e.yml`, …)
Output formats	Terminal, JSON, SARIF 2.1.0, HTML report, GitHub step summary

1.5 Major features

1. **18 rules** in 4 categories — Security (6), Reliability (4), Async (5), AI Patterns (3).
2. **4 presets** — `recommended` (low noise), `strict` (all rules at error), `security`, `agent` (5 high-confidence rules for real-time agent loops).
3. **Zero-config CLI** — works without any ESLint config file.
4. **Changed-file scanning** (`ai-guard changed`) using Git merge-base logic.
5. **Baseline** — record existing findings, report only new ones (`--mode stable|strict`).
6. **SARIF 2.1.0** output for GitHub Code Scanning with stable fingerprints.

7. **GitHub Action** with annotations, step summary and outputs.
8. **MCP server** and **Claude Code hooks** for agent workflows.
9. **Confidence tiers** (`high`, `medium`, `low`, `informational`) and `--fail-on` gating.
10. **Inline-suppression policy** (`allow` / `require-reason` / `ignore`) so a comment cannot silently hide a finding.
11. **Check integrity** — flags when a change weakens the project's own safety checks.
12. **Autofixes** on 4 rules; suggestions on 3.

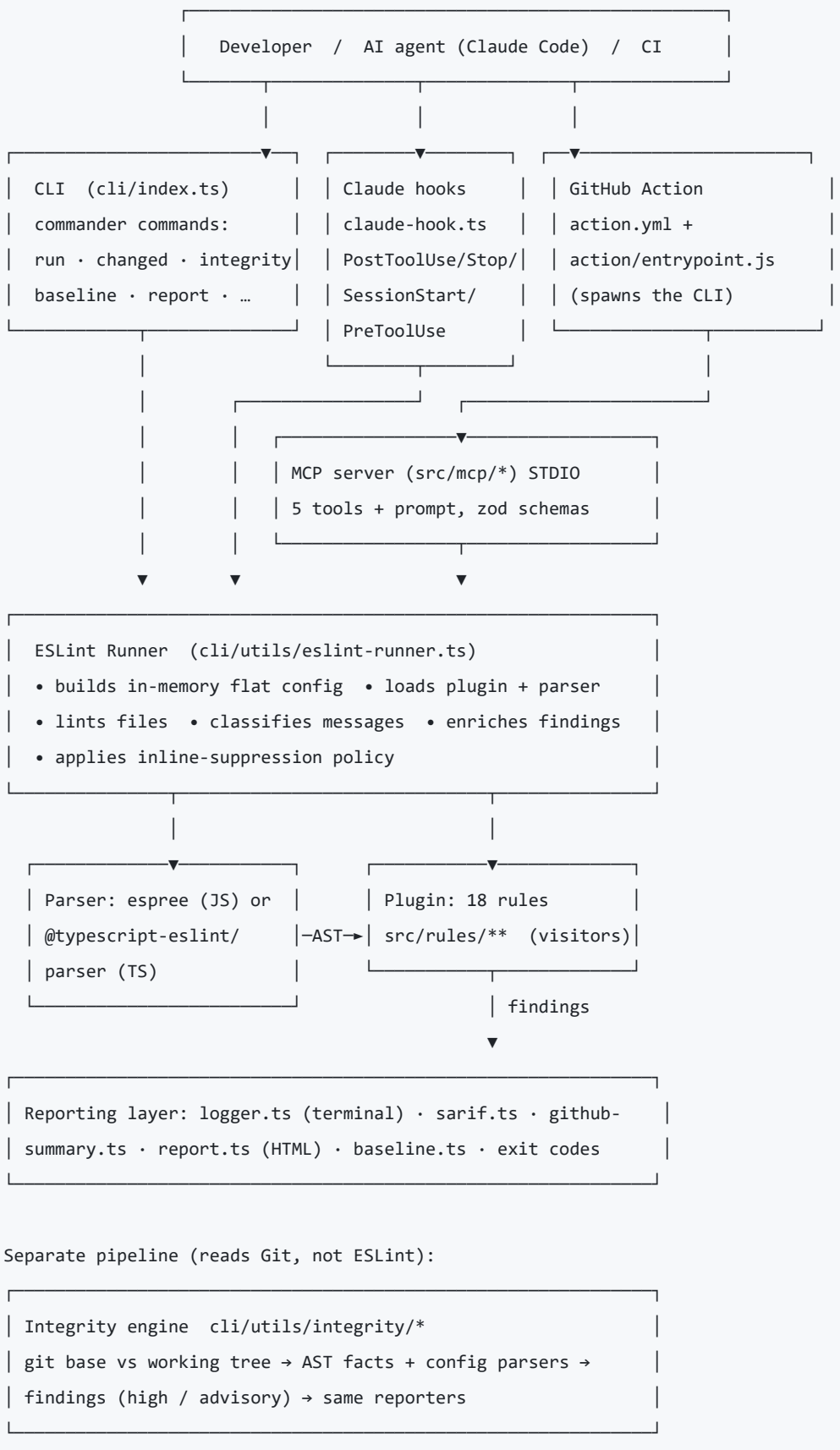
1.6 How the whole project works — input to output

```
INPUT                    PROCESSING                          OUTPUT
─────                    ──────────                          ──────
JS/TS files        ┌──►  1. Resolve files (glob / git diff)     ┌──► Terminal report (grouped, tiers)
(whole project,    │     2. Build ESLint (flat config, in memory)├──► JSON (--json)
 changed files,    │     3. Parse → AST (espree or TS parser)    ├──► SARIF 2.1.0 (--sarif)
 one edited file,  │     4. Run 18 rules (visitor pattern)       ├──► GitHub annotations + step summary
 MCP/hook request) │     5. Classify messages (ai-guard / parser ├──► Exit code (--fail-on)
                   │        error / ecosystem)                   ├──► Hook JSON for Claude / MCP result
                   └──   6. Enrich: confidence, category, fix    └──► Autofix (ESLint --fix semantics)
                         7. Apply inline-suppression policy
                         8. (optional) integrity analysis vs Git base
```

---

2. High-Level Design (HLD)

2.1 System architecture



2.2 Main components and responsibilities

Component	Responsibility	Key files
Plugin entry	Exports `rules` (`allRules`) and `configs` (4 presets)	`src/index.ts`, `src/rules/index.ts`, `src/configs/*.ts`
Rules	Each rule = a `create(context)` returning AST visitors; reports problems with message IDs	`src/rules/{security,reliability,async,ai-patterns}/*.ts`
ESLint runner	Single place that runs ESLint programmatically; normalises results into `RunResult`	`cli/utils/eslint-runner.ts`
Commands	User-facing CLI actions	`cli/commands/*.ts`
Git diff	Finds changed files (PR / staged / uncommitted) with multiple fallback strategies	`cli/utils/git-diff.ts`
SARIF generator	Converts `RunResult` to SARIF 2.1.0 that GitHub accepts	`cli/utils/sarif.ts`
GitHub summary	Annotations, step summary markdown, `GITHUB_OUTPUT`	`cli/utils/github-summary.ts`
Baseline	Save/load/filter findings; integrity acceptances	`cli/utils/baseline.ts`, `cli/commands/baseline.ts`
Directives parser	Parses `eslint-disable…` and `@ts-…` comments	`cli/utils/directives.ts`
Integrity engine	Compares a Git base with the working tree to find weakened checks	`cli/utils/integrity/*`
MCP server	Exposes scanning as tools to AI agents, with input validation and limits	`src/mcp/*`
Action wrapper	Maps Action inputs → CLI args, reads SARIF → Action outputs	`action/entrypoint.js`, `action.yml`

2.3 How components communicate

- **In-process function calls.** CLI commands call `runEslint(options)` and receive a typed `RunResult`. The MCP tools and the Claude hook call the same function (MCP with `isolated: true`).
- **Child process.** The GitHub Action entrypoint calls `childProcess.spawnSync(exe, args, { shell: false })` to run the CLI; it communicates back through the SARIF file and the exit code.
- **Files.** SARIF file (`ai-guard-results.sarif`), `GITHUB_STEP_SUMMARY`, `GITHUB_OUTPUT`, `.ai-guard-baseline.json`, `.claude/settings.json`.
- **STDIO JSON-RPC.** The MCP server uses `StdioServerTransport`; all diagnostics go to stderr so stdout stays pure protocol.
- **Hook protocol.** Claude Code sends hook JSON on stdin; AI Guard answers with one JSON object on stdout and always exits 0.
- **Git.** `git` is invoked with `execFileSync` and argument arrays (never a shell string).

2.4 End-to-end data flow (CLI `run`)

```
user: ai-guard run --path src --strict --sarif
  │
  ▼  cli/index.ts (commander) → run.ts action handler
  │    validate options → pick preset
  ▼
  runEslint({ preset, targetPath })                [cli/utils/eslint-runner.ts]
  │    1. getOrBuildRunner(): load ESLint + plugin + TS parser in parallel (cached)
  │    2. flat config blocks: [ignores] [JS espree] [TS parser]
  │    3. eslint.lintFiles(patterns)                ← ESLint parses → AST → runs rules
  │    4. for each message: classifyMessage → issue | parser-error | ecosystem
  │    5. enrich IssueDetail (confidence, category, remediation)
  │    6. applySuppressionPolicy(directives)
  ▼  RunResult { files[], totalErrors, totalWarnings, ruleBreakdown, … }
  │
  ├─► buildSarifLog(result) → sarifToJson → stdout or file
  ├─► emitGitHubAnnotations / writeGitHubSummary / writeGitHubOutputs (in CI)
  └─► process.exit( resolveExitCode(result, failOn) )
```

2.5 Deployment and delivery view

Surface	How a user gets it
ESLint plugin / CLI	`npm install --save-dev eslint-plugin-ai-guard`, `npx ai-guard run`
GitHub Action	`uses: ai-guard-dev/eslint-plugin-ai-guard@v1` (`action.yml`, `runs: node20`)
MCP server	`ai-guard-mcp` binary (`bin` in `package.json`); Claude plugin under `plugins/ai-guard/`
Docs site	VitePress (`docs/`)

---

3. Low-Level Design (LLD)

3.1 Plugin structure (`src/`)

```
src/
├── index.ts            plugin object { meta, rules, configs } — default export + named exports
├── rules/
│   ├── index.ts        imports all 18 rules → export const allRules = { 'no-empty-catch': noEmptyCatch, …
}
│   ├── security/       no-hardcoded-secret · no-eval-dynamic · no-sql-string-concat
│   │                   no-unsafe-deserialize · require-auth-middleware · require-authz-check
│   ├── reliability/    no-empty-catch · no-broad-exception · no-catch-log-rethrow · no-catch-without-use
│   ├── async/          no-floating-promise · no-async-array-callback · no-await-in-loop
│   │                   no-async-without-await · no-redundant-await
│   └── ai-patterns/    no-dead-branch · no-duplicate-logic-block · no-console-in-handler
├── configs/            recommended.ts · strict.ts · security.ts · agent.ts   (ClassicConfig objects)
└── mcp/                MCP server (create-server.ts, tools/, utils/)
```

`src/index.ts` (real code, shortened):

```
const plugin = {
  meta: { name: 'eslint-plugin-ai-guard', version: pkgVersion },
  rules: allRules,
  configs: { recommended, strict, security, agent },
};
export default plugin;
export { allRules as rules };
export { recommended, strict, security, agent };
```

The version is read from `package.json` at load time with a hard-coded fallback (`let pkgVersion = '1.4.3'`), wrapped in `try/catch` so the plugin still loads in both CJS and ESM.

3.2 Anatomy of a rule

Every rule follows the same shape (shown with the real `no-empty-catch`):

```
const createRule = ESLintUtils.RuleCreator(
  (name) => `https://github.com/ai-guard-dev/eslint-plugin-ai-guard/blob/main/docs/rules/${name}.md`
);

export const noEmptyCatch = createRule({
  name: 'no-empty-catch',
  meta: {
    type: 'problem',
    docs: { description: '…' },
    fixable: 'code',            // allows --fix
    hasSuggestions: true,       // allows editor "suggestions"
    schema: [],                 // no options
    messages: { emptyCatch: '…', addTodoHandler: '…' },   // referenced by messageId
  },
  defaultOptions: [],
  create(context) {
    return {
      CatchClause(node) {                     // ← visitor: called for every catch clause
        if (node.body.body.length === 0) {      // zero statements inside { }
          const comments = context.sourceCode.getCommentsInside(node.body);
          if (comments.length > 0) return;      // a comment = intentional → don't flag
          context.report({
            node,
            messageId: 'emptyCatch',
            fix: (fixer) => fixer.replaceText(node.body, '{ /* TODO: handle error */ }'),
            suggest: [{ messageId: 'addTodoHandler',
                        fix: (fixer) => fixer.replaceText(node.body, '{ /* TODO: handle error */ }') }],
          });
        }
      },
    };
  },
});
```

Part	Purpose
`RuleCreator(url)`	Wraps the rule and auto-fills `meta.docs.url` for each rule
`meta.messages`	Message IDs → text; tests and reports reference IDs, not strings
`meta.schema`	JSON-schema for rule options (only `require-auth-middleware`, `no-console-in-handler`, `no-async-without-await`, `no-await-in-loop` define options in the repo)
`create(context)`	Called once per file; returns a map of **node type → handler**
`context.sourceCode`	Gives text, comments, scope, ancestors (`getCommentsInside`, `getText`, `getAncestors`)
`context.report`	Emits a finding; optional `fix` / `suggest`
`context.filename`	Used by `require-auth-middleware` to skip Electron/script files

3.3 Rule registry and presets

`src/rules/index.ts` imports every rule and exports `allRules`. `src/configs/recommended.ts` maps rule names to severities, e.g.:

```
'ai-guard/no-empty-catch': 'error',
'ai-guard/no-floating-promise': 'error',
'ai-guard/no-hardcoded-secret': 'error',
'ai-guard/no-eval-dynamic': 'error',
'ai-guard/no-sql-string-concat': 'warn',
'ai-guard/no-redundant-await': 'off',
```

`src/configs/agent.ts` exports a single constant `AGENT_RULES` (5 rules at `error`: secret, eval, empty-catch, sql-concat, floating-promise) that the CLI runner **imports** so there is one source of truth.

The CLI runner (`cli/utils/eslint-runner.ts`) keeps its own preset tables `RECOMMENDED_RULES`, `STRICT_RULES`, `SECURITY_RULES` and gets `AGENT_RULES` from `src/configs/agent.ts`. *(Interview note: the plugin's `configs/*.ts` and the runner's tables must stay in sync — a reasonable follow-up concern to raise yourself as a maintenance risk.)*

3.4 The ESLint runner (`cli/utils/eslint-runner.ts`) — the core of the CLI

Important pieces:

Symbol	What it does
`RunOptions`	`{ preset, targetPath, files?, isolated?, inlineSuppressions?, … }`
`RunResult`	`{ files[], totalErrors, totalWarnings, totalIssues, filesScanned, ruleBreakdown, topFiles, ecosystemIssues, parserErrors, ignoredFiles, suppressedIssues?, integrity? … }`
`IssueDetail`	One finding: `ruleId, severity (1|2), message, line, column, confidence, category, remediation, suppression?`
`ISSUE_CONFIDENCE`, `ISSUE_CATEGORY`, `ISSUE_REMEDIATION`, `ISSUE_ASYNC_RISK_TYPE`	Lookup tables that enrich a raw ESLint message
`DEFAULT_IGNORE_PATTERNS`	Globs always skipped (`node_modules`, `dist`, `build`, `.next`, `coverage`, `generated`, …)
`getOrBuildRunner()`	Loads ESLint, the plugin and the TS parser **in parallel** (`Promise.all`), builds the config, and caches the ESLint instance per `cwd::preset::isolated`
`loadPluginModuleFromCwd()`	Tries: the project's installed plugin → `cwd/dist/index.js` → `cwd/src/index.ts` → AI Guard's own package. This is why `npx ai-guard run` works in a project that never installed the plugin

Symbol	What it does
`classifyMessage()`	Decides if an ESLint message is an AI Guard finding, a **parser error**, or an **ecosystem issue** (foreign rule / missing rule). Only AI Guard findings count
`applySuppressionPolicy()`	Applies `eslint-disable` comments according to the chosen mode
`runEslint()`	The main function: resolves files, lints, processes messages, returns `RunResult`

The flat config built in memory (simplified from the real code):

```
const configBlocks = [
  { ignores: DEFAULT_IGNORE_PATTERNS },                        // 1. skip junk early
  { files: ['**/*.js','**/*.jsx','**/*.mjs','**/*.cjs'],       // 2. JavaScript → espree + JSX
    plugins: { 'ai-guard': plugin, 'ai-guard-internal': INTERNAL_PLUGIN },
    languageOptions: { parserOptions: { ecmaVersion: 'latest', sourceType: 'module',
                                        ecmaFeatures: { jsx: true } } },
    rules: { ...rules, ...INTERNAL_RULES } },
  { files: ['**/*.ts','**/*.tsx','**/*.mts','**/*.cts'],       // 3. TypeScript → TS parser if found
    languageOptions: { parser: tsParser, … }, … },
];
const eslint = new ESLint({
  cwd: eslintCwd,
  overrideConfigFile: true,        // ignore the user's eslint config entirely
  overrideConfig: configBlocks,
  allowInlineConfig: false,        // comments cannot disable rules inside ESLint; see 3.9
});
```

Design decisions visible here:

1. **`overrideConfigFile: true`** — the scan never depends on (or breaks because of) the user's ESLint config. This is how "zero-config" works.
2. **Parser fallback** — no `@typescript-eslint/parser`? TypeScript files fall back to espree (may partially fail on TS syntax, and the CLI warns).
3. **Ecosystem vs findings** — messages about foreign plugins never inflate AI Guard's counts.
4. **Cache** — `_runnerCache` is a `Map`, so repeated calls in one process (MCP server, tests) skip the 0.5–1 s ESLint start-up cost.

3.5 CLI structure (`cli/`)

`cli/index.ts` creates one `commander` `Command` and calls `registerRunCommand(program)`, `registerChangedCommand(program)`, … Each command file exports a `register…Command` function — a simple **registration pattern** that keeps commands independent.

Command	File	Purpose
`run`	`commands/run.ts`	Full scan; `--json`, `--sarif`, `--fail-on`, `--max-warnings`, `--inline-suppressions`

Command	File	Purpose
`changed`	`commands/changed.ts`	Scan only changed files; `--pr`, `--staged`, `--base`, `--baseline`, `--integrity`
`integrity`	`commands/integrity.ts`	Detect weakened checks; `--base`, `--enforce`, `--accept <selection>`
`baseline`	`commands/baseline.ts`	`--save` / `--check`, `--mode stable|strict`
`report`	`commands/report.ts`	Standalone HTML report with score
`init`	`commands/init.ts`	Detect ESLint setup and write config
`doctor`	`commands/doctor.ts`	Diagnose ESLint / parser / plugin issues
`preset`, `ignore`	`commands/preset.ts`, `ignore.ts`	Switch preset / add ignore patterns
`init-context`	`commands/init-context.ts`	Generate `CLAUDE.md`, `.cursorrules`, Copilot instructions
`init-claude`	`commands/init-claude.ts`	Write Claude Code hooks to `.claude/settings.json`
`claude-hook`	`commands/claude-hook.ts`	Hook entry point (reads stdin)

Exit codes (`resolveExitCode` in `run.ts`) — `--fail-on` levels: `errors` (default for `run`), `high`, `medium`, `any`, `none`. The tier of each finding comes from `CONFIDENCE_TIER` in `logger.ts`. `run --path <file>` exits **2** when the file is ignored and nothing was scanned.

3.6 Changed-file scanning (`cli/utils/git-diff.ts`)

`getChangedFiles()` tries strategies in order (`tryPrDiff`):

1. `git merge-base HEAD origin/<branch>` then `git diff --name-only <merge-base>`
2. same with the local branch
3. `git diff origin/<branch>...HEAD`
4. `git diff <branch>...HEAD`
5. last resort: `git diff HEAD~1` (shallow-clone safety net)

It also returns `mode: 'fallback'` when Git is unavailable, and the caller does a full scan instead. The file header documents two historical bugs that were fixed: an **operator-precedence bug** (`base ?? env ? … : …`) and a **double `origin/` prefix**.

`getStrictChangedFiles()` is the stricter variant used by MCP: NUL-delimited output (`-z`), validated ref regex, no implicit fallback — so Git failure is never confused with "no changes".

3.7 SARIF generator (`cli/utils/sarif.ts`)

Pipeline: `buildSarifLog(result, version, repoRoot)` → rule descriptors (`buildRuleDescriptors`) + results (`buildSarifResult`) → `sanitizeSarifLog` → `normalizeSarifForGitHub` → `sarifToJson`.

Key functions:

Function	Role
`normalizeSarifPath`	Converts to repository-relative POSIX path (strips drive letters, backslashes, `./`, `GITHUB_WORKSPACE` prefix); **no** `uriBaseId`
`generateStableFingerprint`	`sha256( ruleId : lower(uri) : startLine : normalizedMessage )`
`sanitizeSarifTags`	De-duplicates tags (GitHub requires unique items)
`confidenceToSarifLevel / SecuritySeverity / Precision`	Maps tiers to `error/warning/note`, `security-severity`, `precision`
constants	`SARIF_AUTOMATION_ID = 'ai-guard'`, `FINGERPRINT_KEY = 'ai-guard/v1'` — `CLAUDE.md` says never change them (would orphan existing alerts)

Rules enforced: `startLine >= 1`, `level ∈ {error, warning, note, none}`, `kind ∈ {fail, open, informational}`.

3.8 GitHub Action (`action.yml`, `action/entrypoint.js`)

- `runs: { using: 'node20', main: 'action/entrypoint.js' }` — a JavaScript Action, no Docker.
- Inputs: `preset`, `fail-on`, `changed-only`, `path`, `upload-sarif`, `github-summary`, `working-directory`, `package-manager`, `sarif-output`, `install-deps`, `baseline`, `integrity`, `inline-suppressions`.
- Flow in `main()`:
  1. Read inputs with `getInput()`.
  2. Detect package manager (`pnpm-lock.yaml` / `yarn.lock` / npm) and optionally install dependencies.
  3. Build `cliArgs` **as an array** (`changed` or `run`, `--pr`, `--path`, preset flags, `--fail-on`, `--sarif-output`, `--sarif`, optional `--baseline`, `--integrity`, `--inline-suppressions`).
  4. Run `spawnSync(exe, args, { shell: false })` — local `node_modules/.bin/ai-guard` if present, otherwise `npx --yes -p eslint-plugin-ai-guard ai-guard …`.
  5. Parse the SARIF file to compute outputs (`issues-found`, `high-confidence-count`, `medium-confidence-count`, `files-scanned`, …) — integrity results are counted separately.
  6. Exit with the CLI's code; `null` status (signal kill) is treated as failure. `fail-on: none` returns 0 **unless** `integrity: enforce` is on.
- Security note (tests: `tests/ci/entrypoint-security.test.ts`): arguments are passed as a structured array with `shell: false`, so a path like `src;echo PWNED` stays one argument.

3.9 Inline-suppression design (`cli/utils/directives.ts` + runner)

Problem found and fixed in 1.4.3: an `// eslint-disable-next-line ai-guard/no-empty-catch` comment made findings **disappear silently**. Design now:

1. ESLint is created with `allowInlineConfig: false`, so ESLint itself never hides AI Guard findings.
2. An internal rule (`ai-guard-internal/directives`, `directiveCollectorRule`) reads comments from the real parse (so text inside strings is ignored) and stores parsed directives per file.
3. `applySuppressionPolicy(issues, directives, mode)` decides per finding using `findCoveringDirective()`:

- `allow` (CLI default) — hides, but the hidden finding is **listed** (`suppressedIssues`, step summary, SARIF run property).
- `require-reason` — hides only if the directive has `-- reason` (used by the Claude hook).
- `ignore` — never hides (used by MCP).

3.10 MCP server (`src/mcp/`)

- `server.ts`: reads `AI_GUARD_WORKSPACE` (or `process.cwd()`), `createMcpServer(workspace)`, connects `StdioServerTransport`, prints "Server started" to **stderr**.
- `create-server.ts`: `McpServer` from `@modelcontextprotocol/sdk`; each tool is registered with `server.registerTool(name, { description, inputSchema (zod), outputSchema (zod), annotations: { readOnlyHint: true } }, handler)`.
- Tools: `ai_guard_scan_file`, `ai_guard_scan_diff`, `ai_guard_scan_project`, `ai_guard_check_integrity`, `ai_guard_rules`; prompt `verify_changes`.
- Reliability design:
  - A `busy` flag rejects overlapping calls (`SERVER_BUSY`).
  - Result contract: `success`, `status` (`complete|partial|failed`), `clean`, `errors[{code,message}]`. **Only `status: complete` + `clean: true` is a clean scan.**
  - Limits in `utils/scan.ts`: 500 files, 1 MiB per file, 20 MiB total, `maxIssues` default 50 (max 1000).
  - Path validation in `utils/workspace.ts` (no escaping the workspace, no symlinks in project scans).
  - Scans use `isolated: true` — bundled rules/parser only, never code from the scanned workspace.

3.11 Claude Code hooks (`cli/commands/claude-hook.ts`, `init-claude.ts`)

`ai-guard claude-hook` reads one JSON object from stdin and dispatches on `hook_event_name`:

Event	Behaviour
`PostToolUse` (Edit/MultiEdit/Write)	Lint the edited file (`agent` preset, `require-reason` policy) + per-file integrity check → JSON `{ decision: "block", reason }` and/or `additionalContext`
`SessionStart`	Snapshot the working tree as the integrity base (temporary Git index, user files untouched)
`Stop`	Integrity check of the whole session; high findings block finishing **once per unique finding set** (state file keyed by session), later only `systemMessage`
`PreToolUse` (Bash)	Ask before `git commit --no-verify`, `HUSKY=0`, or an agent running `ai-guard integrity --accept`

It **always exits 0** — the documented protocol delivers feedback through JSON, and plain stdout from an exit-0 hook is not shown to Claude (this was the bug fixed in 1.4.3; see `CHANGELOG.md` and `docs/integrations/claude-code.md`).

3.12 Integrity engine (`cli/utils/integrity/`) — new in 1.4.3

File	Role
`git.ts`	`resolveBase`, `listChanges`, `readBlobs` (batched `git cat-file`), `addedLinesForAll`; every call uses `execFileSync` with argument arrays

File	Role
`js-facts.ts`	Parses JS/TS with ESLint's `Linter` + the same parsers; extracts directives, test declarations (`it/test/describe`, `.skip`, `.only`, `xit`, `fit` …), assertion counts, `as any` casts
`analyze-js.ts`	Suppression and test analyzers (multiset comparison base vs head)
`analyze-config.ts`	ESLint config, `tsconfig`, coverage thresholds, `package.json` scripts, GitHub workflows, husky hooks, snapshots
`commands.ts`	Recognises weakening shell patterns (`
`engine.ts`	`analyzeIntegrity()` orchestrates everything → `IntegrityReport`
`types.ts`	Finding model, rule catalog (`INTEGRITY_RULES`), fingerprints, tiers (`high` vs `advisory`)
`report.ts`	Renders the same findings to terminal / JSON / Markdown / annotations / SARIF / agent text

Only **high-confidence** findings can fail a check, and only when enforcement is explicitly enabled (`--integrity enforce` / `integrity: enforce`).

3.13 LLD of the `no-hardcoded-secret` rule (deep dive)

File: `src/rules/security/no-hardcoded-secret.ts` (221 lines). Severity: `error` in `recommended`, `strict`, `security`, `agent`. Confidence tier: **high**. Autofix: yes.

3.13.1 What it detects

A string literal assigned to something **whose name looks like a secret**:

```
const apiKey = 'sk-1234567890abcdef1234567890abcdef';      // variable
config.clientSecret = 'abcdef1234567890abcdef1234567890';   // assignment to a member
const cfg = { jwtSecret: 'my-super-secret-jwt-key-that-is-long' };   // object property
```

It does **not** analyse the *value's* format (no prefix detection like `AKIA…`, no entropy).

3.13.2 Building blocks (real constants)

```
const SECRET_NAME_PATTERN =
  /(?:secret|password|passwd|api[_-]?key|auth[_-]?token|access[_-]?token|private[_-]?key|client[_-]?
secret|jwt[_-]?secret|encryption[_-]?key|signing[_-]?key)/i;

const FALSE_POSITIVE_VALUES = new Set(['password','secret','token','key','api_key','apikey','',
  'undefined','null','test','example','placeholder','changeme','your-api-key','your-
secret','xxx','todo']);

const NON_SECRET_NAME_SUFFIXES =
  /(?:hash|hashed|digest|checksum|encrypted|encoded|hmac|bcrypt|argon|salted)$/i;
```

Constant	Purpose
`SECRET_NAME_PATTERN`	Only names that suggest secrets are considered. Case-insensitive; `api_key`, `api-key`, `apiKey` all match
`FALSE_POSITIVE_VALUES`	Well-known placeholders (compared lowercase, `_` → `-`, so `YOUR_API_KEY` ≡ `your-api-key`)
`NON_SECRET_NAME_SUFFIXES`	`passwordHash`, `tokenDigest` … are hashes, not plaintext secrets

3.13.3 Three visitors, one decision pipeline

```
AST node ─┬─ VariableDeclarator   const apiKey = '…'
          ├─ AssignmentExpression obj.apiKey = '…'      (left must be a MemberExpression)
          └─ Property             { apiKey: '…' }       (key must be an Identifier)
                  │
                  ▼  same checks, in this order
  1. name matches SECRET_NAME_PATTERN?                 no → return
  2. value is a plain string?  (Literal string, or TemplateLiteral with 0 expressions)   no → return
  3. value is a known placeholder?  (isFalsePositiveValue)                              yes → return
  4. value.length >= 8 ?                                                                 no  → return
  5. name ends with hash/digest/…?  (NON_SECRET_NAME_SUFFIXES)                           yes → return
  6. context.report({ node: valueNode, messageId: 'hardcodedSecret',
                      data: { name, envName: toEnvVarName(name) }, fix })
```

The visitor for variables (real code):

```
VariableDeclarator(node) {
  if (!node.init) return;
  if (!node.id || node.id.type !== AST_NODE_TYPES.Identifier) return;   // skips destructuring
  const varName = node.id.name;
  if (!SECRET_NAME_PATTERN.test(varName)) return;
  const value = getStringValue(node.init);        // string | null
  if (value === null) return;                     // process.env.X, function calls, numbers…
  if (isFalsePositiveValue(value)) return;
  if (value.length < 8) return;
  if (NON_SECRET_NAME_SUFFIXES.test(varName)) return;
  context.report({
    node: node.init,
    messageId: 'hardcodedSecret',
    data: { name: varName, envName: toEnvVarName(varName) },
    fix: (fixer) => fixer.replaceText(node.init, getProcessEnvAccessText(toEnvVarName(varName))),
  });
}
```

3.13.4 Helper functions

Helper	What it does	Example
`getStringValue(node)`	Returns the string for a string `Literal` or a no-expression `TemplateLiteral`; otherwise `null`	`'abc'` → `"abc"`; `` `abc${x}` `` → `null`
`isFalsePositiveValue(v)`	lowercases, replaces `_` with `-`, checks the set	`YOUR_API_KEY` → true
`toEnvVarName(name)`	camelCase → UPPER_SNAKE	`jwtSecret` → `JWT_SECRET`, `clientSecret` → `CLIENT_SECRET`, `apiKey` → `API_KEY`
`getProcessEnvAccessText(env)`	`process.env.NAME`, or `process.env['NAME']` if not a valid identifier	
`isRuleMetaMessagesProperty(node)`	Skips `Property` nodes inside a `messages: { … }` object	prevents AI Guard from flagging *its own* rule message `hardcodedSecret: '…'`

The last helper exists because the project **scans itself** in CI (strict preset). Rule message IDs like `hardcodedSecret` match the name pattern and their text is longer than 8 characters. Test case "ESLint rule meta message IDs are not secrets" in `tests/rules/no-hardcoded-secret.test.ts` guards it.

3.13.5 Autofix

`fixer.replaceText(valueNode, 'process.env.API_KEY')` — test expectation:

```
before:  const apiKey = 'sk-1234567890abcdef1234567890abcdef';
after:   const apiKey = process.env.API_KEY;
```

Trade-off: the fix changes runtime behaviour — the app now needs `API_KEY` defined. It also does **not** remove the secret from Git history. (Both are honest talking points.)

3.13.6 Tests (`tests/rules/no-hardcoded-secret.test.ts`)

Valid cases include: `process.env.API_KEY`; short value `'test'`; non-matching name (`username`); empty string; placeholder `'your-api-key'`; long non-secret name (`description`); `process.env` objects; `'changeme'`; destructuring from `process.env`; and `const PLACEHOLDER = 'INSERT_API_KEY_HERE'`.

> **Interviewer trap:** that last case is valid only because the *name* `PLACEHOLDER` does not match `SECRET_NAME_PATTERN` — the value `INSERT_API_KEY_HERE` is not in the placeholder set. Don't say "the rule recognises placeholder text like INSERT_..."; say "it is skipped because the variable name isn't secret-like".

Invalid cases verify the exact `output` of the fix for `apiKey`, `password`, `jwtSecret`, `authToken`, object property `clientSecret`, and member assignment.

3.13.7 Limitations (be ready to state them)

- Name-driven: `const x = 'sk_live_…'` is **not** flagged (name doesn't suggest a secret). Secrets passed directly as call arguments (`connect('hunter2hunter2')`) are also not flagged.
- Only plain strings: concatenation, template with expressions, or values built at runtime are ignored.
- Length threshold 8 means short real passwords are missed.
- Destructuring patterns (`const { apiKey } = …`) are skipped by design (`id.type !== Identifier`).
- No value-format detection (no provider-prefix regexes, no entropy). *(Possible improvement to mention: combine name check with value-format/entropy checks.)*
- Literal fixes can change behaviour; the secret remains in Git history.

---

4. AST explained with examples

4.1 What is an Abstract Syntax Tree?

An AST is a **tree representation of the structure of source code**. A parser turns text into tokens, then into nodes such as "variable declaration", "function call", "catch clause". Each node has a `type`, child nodes and a location (`loc`, `range`). "Abstract" means punctuation and whitespace are dropped; only meaning-bearing structure remains. ESLint uses the **ESTree** specification for JavaScript; `@typescript-eslint/parser` produces ESTree-compatible nodes plus TypeScript-specific ones (`TSAsExpression`, `TSAnyKeyword`, …).

4.2 Why AST instead of text matching (regex)?

Need	Regex problem	AST solution
Find an empty `catch {}`	`catch\s*\(.*\)\s*\{\s*\}` breaks on comments, nested braces, `catch {` without parameter, strings containing `catch (e) {}`	`CatchClause` whose `body.body.length === 0` — exact, ignores strings/comments
Ignore code in strings/comments	`"it.skip(...)"` in a string looks like code	Comments and strings are separate token/node types
Understand context	Can't know if a call is inside a loop or an async function	Walk ancestors / scope (`getAncestors`, `getScope`)
Resolve a variable	Can't connect `query` to its earlier `const query = 'SELECT …' + id`	Track `VariableDeclarator` nodes; `no-sql-string-concat` keeps a `variableMap`
Safe fixes	A regex replace can corrupt code	`fixer.replaceText(node, …)` edits exact node ranges

4.3 A simple example

```
async function load(id) {
  try {
    const user = await fetchUser(id);
  } catch (e) {}
  sendEmail(id);
}
```

Simplified ESTree (only `type` and key fields; real nodes also carry `range` / `loc`):

```
Program
└─ body[0]: FunctionDeclaration  { id: Identifier(load), async: true, params: [Identifier(id)] }
   └─ body: BlockStatement
      ├─ TryStatement
      │   ├─ block: BlockStatement
      │   │   └─ VariableDeclaration (kind: const)
      │   │       └─ VariableDeclarator
      │   │           ├─ id:   Identifier(user)
      │   │           └─ init: AwaitExpression
      │   │                   └─ argument: CallExpression
      │   │                       ├─ callee: Identifier(fetchUser)
      │   │                       └─ arguments: [Identifier(id)]
      │   ├─ handler: CatchClause
      │   │   ├─ param: Identifier(e)
      │   │   └─ body:  BlockStatement  { body: [] }     ← empty → no-empty-catch fires here
      │   └─ finalizer: null
      └─ ExpressionStatement
          └─ expression: CallExpression                  ← standalone call → no-floating-promise looks
here
              ├─ callee: Identifier(sendEmail)
              └─ arguments: [Identifier(id)]
```

4.4 Nodes AI Guard actually uses (verified in `src/rules/` and `cli/utils/integrity/`)

Node	Meaning	Used by (examples)
`Program`	Root of the file	Directive collector (reads all comments once)
`VariableDeclaration` / `VariableDeclarator`	`const a = …` / one binding with `id` and `init`	`no-hardcoded-secret`, `no-sql-string-concat` (builds `variableMap`)
`AssignmentExpression`	`a.b = …`	`no-hardcoded-secret`, `no-sql-string-concat`
`Property` / `ObjectExpression`	`{ key: value }`	`no-hardcoded-secret`, Fastify option detection in `require-auth-middleware`, config analysis

Node	Meaning	Used by (examples)
`FunctionDeclaration`, `FunctionExpression`, `ArrowFunctionExpression`	Functions (have `.async`)	`no-floating-promise` (`isAsyncFunctionLike`), `no-async-without-await`
`CallExpression` / `NewExpression`	`f(x)` / `new F(x)`	`no-eval-dynamic`, `no-sql-string-concat`, `require-auth-middleware` (`router.get(...)`)
`MemberExpression`	`a.b` (`object`, `property`)	callee shapes: `db.query`, `window.eval`, `Promise.all`
`ExpressionStatement`	A statement that is just an expression	`no-floating-promise`
`TryStatement` / `CatchClause`	`try { } catch (e) { }`	`no-empty-catch`, `no-catch-log-rethrow`, `no-catch-without-use`, `nodeHasCatchClause`
`AwaitExpression`	`await x`	`no-await-in-loop`, `no-redundant-await`, `no-async-without-await`
`BinaryExpression`	`a + b`	`no-sql-string-concat` (`operator === '+'`), `require-authz-check` (`===`)
`TemplateLiteral`	`` `a${b}` `` (`quasis`, `expressions`)	`no-sql-string-concat`, `no-hardcoded-secret` (only when no expressions)
`Literal`	string/number/boolean/null	`no-eval-dynamic` (`isLiteral`), secret rule
`Identifier`	A name	everywhere
`IfStatement`	`if (cond)`	`no-dead-branch` (per the older guide and rule purpose)
`ArrayExpression`, `SpreadElement`	`[a, ...b]`	middleware arrays in `require-auth-middleware`
`TSAsExpression`, `TSAnyKeyword`	`x as any`	integrity `as any` counting

4.5 How a real AI Guard rule visits the AST

Take `no-eval-dynamic` (real code, abbreviated):

```
create(context) {
  return {
    CallExpression(node) {
      if (node.callee.type === AST_NODE_TYPES.Identifier && node.callee.name === 'eval') {
        if (node.arguments.length === 0) return;
        const firstArg = node.arguments[0];
        if (isLiteral(firstArg)) return;                    // eval('1+1') is allowed
        context.report({ node, messageId: 'evalDynamic', data: { callee: 'eval()' } });
        return;
      }
      // also window.eval / globalThis.eval, and Function(...) without `new`
    },
    NewExpression(node) {                                   // new Function(...)
      if (isFunctionConstructorCallee(node.callee)) {
        if (node.arguments.length === 0) return;
        if (node.arguments.every(isLiteral)) return;
        context.report({ node, messageId: 'newFunctionDynamic' });
      }
    },
  };
}
```

For `eval(userInput)` the AST is `CallExpression { callee: Identifier(eval), arguments: [Identifier(userInput)] }`; `Identifier` is not a `Literal`, so it is reported. For `eval("1+1")` the argument is a `Literal`, so it is allowed.

And `no-floating-promise`'s core decision (real code, abbreviated):

```
ExpressionStatement(node) {
  if (node.expression.type !== AST_NODE_TYPES.CallExpression) return;   // only bare calls
  if (isExpressionHandled(node)) return;                                 // .then/.catch/void/await…
  const localAsyncInfo = isLocallyAsyncCallee(callExpr, context);        // scope lookup
  if (localAsyncInfo.isAsync && localAsyncInfo.hasInternalErrorHandling) return;
  if (localAsyncInfo.isAsync || isKnownPromiseFactoryCall(callExpr) || isPromiseLikeByTypeInfo(callExpr,
context)) {
    context.report({ node, messageId: 'floatingPromise',
                     fix: (fixer) => fixer.replaceText(node.expression, `void ${text}`) });
  }
}
```

4.6 ESLint concepts mapped to the project

Concept	Explanation	In AI Guard
Parser	Turns source into an AST (`espree` default; `@typescript-eslint/parser` for TS)	Runner chooses per file group; falls back to espree if TS parser missing

Concept	Explanation	In AI Guard
Visitor	Object whose keys are node types (or selectors); ESLint calls them during one depth-first traversal	All 18 rules return visitors, e.g. `{ CatchClause(node) {…} }`
Rule metadata (`meta`)	`type` , `docs` , `fixable` , `hasSuggestions` , `schema` , `messages`	Shown in every rule
Context (`context`)	Per-file handle: `report` , `sourceCode` , `filename` , `options`	`context.sourceCode.getCommentsInside` , `getAncestors` , `getText` , `context.report`
Scope manager	Tracks variable declarations/references	`no-floating-promise` resolves a callee to a local `async` function through scope chains
Fixer	Produces text edits; ESLint applies non-overlapping fixes	`fixer.replaceText(node, …)` in 4 rules
Flat config	Modern config array (`files` , `rules` , `languageOptions` , `plugins`)	Built in memory by the runner
Selectors	Node-type keys (and CSS-like selectors)	Only node-type keys are used in the repository's rules

5. Technologies and concepts

Term	Simple explanation	Where in AI Guard
ESLint	A pluggable JavaScript linter. It parses code, runs rules, reports problems	Host engine for every rule; the CLI drives ESLint programmatically (`new ESLint(...)` , `lintFiles`)
ESLint plugin	A package that exports `rules` and `configs`	`src/index.ts`
Flat config	ESLint's newer config format (array of objects)	Runner builds one in memory; docs mention legacy `.eslintrc` support too
TypeScript	Typed superset of JavaScript	All source is TS (`strict: true`); rules handle TS nodes; TS parser optional
JavaScript / ESM / CJS	Module systems	tsup produces CJS + ESM for the plugin, CJS for CLI/MCP
AST / ESTree	Tree of code structure	Sections 4.x
Static analysis	Examining code without running it	The entire tool; "deterministic" = same input → same output, no AI model involved
Heuristic	A practical rule that works most of the time	Name patterns, placeholder lists, known promise factories (`fetch` , `Promise.*`)
False positive / negative	Flagging good code / missing bad code	`CONTRIBUTING.md` : "precision over recall"; confidence tiers
Node.js	JS runtime	Requires `>=20` ; CLI, Action and MCP are Node programs
CLI	Command-line interface	`commander` registers commands and options
npm package / `bin`	Distribution unit; `bin` creates executables	`bin: { "ai-guard": …, "ai-guard-mcp": … }` , `files: ["dist", …]`

Term	Simple explanation	Where in AI Guard
tsup	TypeScript bundler built on esbuild	`tsup.config.ts` → 3 bundles; bundles CLI libs so the published package has only one runtime dep
Vitest	Test runner	`npm test`; rule tests use `@typescript-eslint/rule-tester`
RuleTester	Runs `valid` and `invalid` code samples against a rule	`tests/helpers/rule-tester.ts` (shared)
Git merge-base	Common ancestor of two branches	Used to find *only* the PR's changes
GitHub Actions	CI/CD automation on GitHub	`action.yml`, workflows in `.github/workflows/`
SARIF 2.1.0	JSON standard for static-analysis results	GitHub Code Scanning reads it to show alerts in the Security tab
Fingerprint	Stable ID for a finding across runs	`partialFingerprints['ai-guard/v1']`
MCP (Model Context Protocol)	Protocol that lets AI clients call external tools	`src/mcp/`, STDIO JSON-RPC
STDIO transport	Client launches the server as a child process and talks over stdin/stdout	`StdioServerTransport`; logs only on stderr
Zod	Runtime schema validation	MCP input/output schemas
Hooks (Claude Code)	Commands the agent runs on lifecycle events	`init-claude` writes them; `claude-hook` handles them
Baseline	Snapshot of known findings so only new ones are reported	`.ai-guard-baseline.json`, modes `stable` (file+rule+message) and `strict` (+line/col)
Confidence tier	Reliability label for a rule's findings	`CONFIDENCE_TIER` → used by `--fail-on`, SARIF precision, annotations
Autofix vs suggestion	Fix applied by `--fix` vs a choice shown in the editor	`fixable: 'code'` vs `hasSuggestions: true`
Zod / `execFileSync` / `shell:false`	Input validation / running programs without a shell	Avoid shell injection in Git calls and the Action
Monorepo support	Scanning part of a repo	`--path`, `working-directory` Action input

---

6. Expected technical questions and answers

Answers are written to be spoken. Numbers follow Section 0.

A. Project overview and architecture

Q1. What is AI Guard in one sentence? It's an ESLint plugin and toolchain with 18 deterministic rules that catches bugs and security problems that AI coding assistants commonly generate, and it plugs into the terminal, GitHub pull requests, and AI agents.

Q2. What problem does it solve? AI-written code compiles and looks clean, so reviewers trust it, but it often drops `await`, swallows errors in empty `catch` blocks, hardcodes keys, or concatenates SQL. Standard

linters don't have rules for most of these, or need heavy type-checking setup. AI Guard catches them fast and without configuration.

Q3. Does it use an LLM or machine learning? No. It is pure AST static analysis — the same input always gives the same output, with no API cost or latency. The README states it is not an AI detector either; it flags risky patterns regardless of who wrote them.

Q4. Walk me through the architecture. Entry points — the CLI, the GitHub Action, the MCP server and the Claude hook — all call one core, `runEslint` in `eslint-runner.ts`. It builds an in-memory ESLint flat config with our plugin and the right parser, lints files, then classifies and enriches results into a `RunResult`. Reporters turn that into terminal output, JSON, SARIF, annotations, or exit codes.

Q5. Why is the runner a separate layer from the rules? Rules only know about AST nodes. The runner owns everything operational — config, parser selection, ignore patterns, caching, error classification, severity/confidence metadata. That keeps rules small and testable and lets every integration reuse the same engine.

Q6. What does "zero-config" mean technically? The runner creates `new ESLint({ overrideConfigFile: true, overrideConfig: configBlocks })`. `overrideConfigFile: true` tells ESLint not to read the user's config; we supply our own blocks in memory.

Q7. How is the package distributed? As one npm package with two binaries (`ai-guard` , `ai-guard-mcp`) and a plugin entry. `tsup` produces CJS+ESM for the plugin and CJS bundles for the CLI and MCP server, with CLI libraries bundled so only `@typescript-eslint/utils` is a runtime dependency.

Q8. What are the four presets and why? `recommended` (day-one safe: only high-confidence rules at error), `strict` (all rules at error, used to scan its own repo), `security` (security rules only), `agent` (5 high-signal rules for fast feedback inside AI agent loops).

B. HLD and LLD

Q9. Describe the end-to-end data flow for `ai-guard changed` in a pull request. The command reads `GITHUB_BASE_REF` , `git-diff.ts` finds the merge-base and lists changed JS/TS files, only those files are passed to `runEslint({ files })` , results are enriched, then SARIF, annotations, the step summary and outputs are written and the exit code follows `--fail-on` .

Q10. How do the components talk to each other? Mostly direct function calls with a typed `RunResult` . The Action talks to the CLI by spawning a process with an argument array and reading back the SARIF file and the exit code. The MCP server talks to clients over STDIO JSON-RPC. The Claude hook uses stdin/stdout JSON.

Q11. What is `RunResult` and why does it matter? It's the single data contract between the engine and every reporter: files with issues, totals, rule breakdown, ecosystem issues, parser errors, ignored files, suppressed findings, and optionally integrity results. One structure means the terminal, JSON, SARIF and summary can't drift apart.

Q12. What design patterns are used? Visitor (ESLint rules), registration pattern for CLI commands (`registerXCommand`), factory (`RuleCreator`), strategy (several fallback strategies for Git diff), module-level cache (`_runnerCache`), and a pipeline for SARIF normalization.

Q13. How does the plugin find itself when run via `npx`? `loadPluginModuleFromCwd` tries the project's installed copy, then local `dist` / `src`, then the package that contains the CLI. That fallback order is why the CLI works even where the plugin isn't installed locally. A bug in 1.4.3 (the CJS build replacing `import.meta` with `{}`) had broken that last fallback; it's fixed by `shims: true` in `tsup.config.ts` with a regression test (`tests/cli/standalone-cli.test.ts`).

C. AST and ESLint internals

Q14. What is an AST and why not regex? An AST is a tree of the code's structure. Regex can't reliably tell code from comments or strings, can't see nesting, and can't resolve variables. For example, `try { run(); } catch (e) {}` is one `CatchClause` with an empty body array; a regex would have to guess braces.

Q15. How does ESLint run a rule? ESLint parses the file to an AST, then calls each rule's `create(context)` once. The returned object maps node types to functions. ESLint walks the tree once and calls the matching function when it enters each node. Rules call `context.report` to emit problems.

Q16. How does `no-empty-catch` work? It listens for `CatchClause`. If `node.body.body.length === 0` it checks `context.sourceCode.getCommentsInside(node.body)`. A comment means the developer documented the intent, so it isn't flagged. Otherwise it reports and offers an autofix that inserts `{ /* TODO: handle error */ }` plus an editor suggestion.

Q17. How does `no-floating-promise` decide a call is a promise without type info? Three signals: the callee is a local `async` function found via scope lookup; the call is a known promise factory (`fetch`, or `Promise.resolve/all/…`); or, when TypeScript parser services exist, the type checker says the result is a Promise. If a local async helper already has its own `try/catch`, it is treated as intentionally fire-and-forget and not flagged.

Q18. What does `context.sourceCode.getScope` / scope analysis give you? Variable resolution: from an identifier I can find where it was declared and whether the init is an async function. That's how the rule answers "is `saveProfile` async?" in the same file.

Q19. What is the difference between `fix` and `suggest`? `fix` is applied automatically with `eslint --fix`, so it must be safe. `suggest` is shown to the user to choose. `no-empty-catch` uses both; four rules have `fix`.

Q20. Why does AI Guard disable ESLint's inline config in its own runner? Because `// eslint-disable-next-line ai-guard/…` made findings vanish without a trace. The runner now uses `allowInlineConfig: false`, parses directives itself (`directives.ts`), and applies a policy (`allow`, `require-reason`, `ignore`) that still reports what was hidden.

D. TypeScript and JavaScript code

Q21. Why `AST_NODE_TYPES` instead of string literals? It's a typed enum from `@typescript-eslint/utils`. The compiler then checks node-type names and narrows the node type: after `node.type === AST_NODE_TYPES.Identifier`, TypeScript knows `node.name` exists.

Q22. What does `fixer.replaceText(node, text)` do? It returns an edit that replaces the exact source range of that node. ESLint collects fixes from all rules and applies non-overlapping ones; if fixes overlap, later passes handle the rest.

Q23. Explain the TypeScript narrowing in `getStringValue`. It takes an expression and returns `string | null`: a `Literal` whose `typeof value === 'string'`, or a `TemplateLiteral` with zero expressions and one quasi. The return type forces callers to handle `null`.

Q24. Why does the CLI use `execFileSync` with an argument array for Git? No shell is involved, so a branch name or filename with `;`, spaces or quotes can't inject commands. Refs are also validated by a strict regex and `--end-of-options` is used with `rev-parse`.

Q25. How is the project kept type-safe across three tsconfigs? `tsconfig.json` covers `src/`, `tsconfig.mcp.json` covers the MCP server plus the files it imports, and `tsconfig.cli.json` covers `cli/`. `npm run typecheck` now runs all three; before 1.4.3 the CLI config had 14 unused-declaration/type errors that were hidden because it wasn't part of the script.

E. Individual rules

Q26. Explain `no-hardcoded-secret` end to end. Three visitors — `VariableDeclarator`, `AssignmentExpression`, `Property` — apply the same filter chain: name matches a secret regex; value is a plain string; not a placeholder like `changeme`; length at least 8; name doesn't end with `hash`/`digest`/etc. If all pass, it reports and the fix replaces the literal with `process.env.<NAME>` computed by `toEnvVarName` (`jwtSecret` → `JWT_SECRET`). It has no entropy or value-format detection, only name and value-shape rules.

Q27. How does the secret rule avoid flagging itself? `isRuleMetaMessagesProperty` skips properties inside a `messages` object, since AI Guard's own message IDs (like `hardcodedSecret`) match the name regex and are long strings. This matters because the repo scans itself in CI.

Q28. How does `no-sql-string-concat` work? It keeps a `variableMap` of identifier → initializer built from `VariableDeclarator` and `AssignmentExpression`. On a `CallExpression` that looks like a SQL sink (`query`, `execute`, …), it resolves the first argument through the map and checks for a `+` concatenation or a template literal with expressions that includes SQL-like text. It skips known query-builder chains to avoid false positives.

Q29. Why is `no-eval-dynamic` allowed to ignore `eval('literal')`? There's no injection if the argument is a static literal. The rule only reports when the first argument isn't a `Literal` (or a no-expression template). It also covers `window.eval`, `globalThis.eval`, `new Function(...)`, and `Function(...)` called without `new`.

Q30. What is special about `require-auth-middleware`? It recognises Express/Fastify route registrations and checks whether an auth middleware appears before the handler. It tracks router-level auth (`router.use(auth)`, scoped mount paths, Fastify `addHook`) per receiver name so one router's auth doesn't leak to another, treats public routes by exact path segment (`/health`, `/login`), and skips Electron/scripts files. In 1.4.3 it was made segment-exact: before, `/authors` was treated as public because it started with `/auth`.

Q31. Which rules have autofix and why not more? Four: `no-await-in-loop`, `no-floating-promise`, `no-empty-catch`, `no-hardcoded-secret`. Security rules that change data flow (SQL, auth) aren't auto-rewritten because a wrong guess could break behaviour — e.g., parameter placeholders differ between database drivers.

F. CLI and GitHub Actions

Q32. What does `--fail-on` do? It selects which findings produce a non-zero exit: `errors` (severity 2), `high`, `medium`, `any` (anything not informational), or `none` (report only). Tiers come from `CONFIDENCE_TIER`.

Q33. How does `changed` avoid scanning the whole repo? It computes the merge-base with the PR's base branch and lists changed files, then passes that explicit list to ESLint (`files:` option) so ESLint skips glob traversal.

Q34. What happens when Git isn't available or the base is shallow? `getChangedFiles` returns `mode: 'fallback'` and the command falls back to a full scan; it tries five strategies in order including a `HEAD~1` safety net. The strict variant used by MCP and integrity returns errors instead of guessing.

Q35. Walk me through the GitHub Action. `action.yml` declares inputs and runs `action/entrypoint.js` on Node 20. The entrypoint maps inputs to CLI arguments as an array, runs the CLI with `shell: false` (local binary or `npx`), parses the SARIF file to set Action outputs, and returns the CLI's exit code. SARIF is uploaded by a separate `github/codeql-action/upload-sarif` step in the workflow.

Q36. Why SARIF rather than only annotations? Annotations from workflow commands are transient. SARIF uploaded to Code Scanning creates persistent, trackable alerts in GitHub's Security tab. `CLAUDE.md` documents the constraints: repository-relative POSIX paths, no `uriBaseId`, constant `automationDetails.id`, and a deterministic `partialFingerprints` key.

Q37. How are alerts kept stable across runs? Three anchors: `automationDetails.id = "ai-guard"`, `partialFingerprints["ai-guard/v1"]` (SHA-256 of rule, lower-cased path, line and normalised message), and a fixed `category` on the upload step. Note the line number is part of the hash, so a finding that moves lines is treated as a new fingerprint — a known trade-off.

G. MCP integration

Q38. What is MCP and why add it? Model Context Protocol lets AI clients call tools. The hook is reactive (runs after edits); MCP is proactive — the agent can scan a file, a diff or the project, look up rule docs, or check integrity before declaring work done.

Q39. How is the MCP server made safe and reliable? Zod input/output schemas; read-only tool annotations; workspace path validation and symlink refusal; hard limits (500 files, 1 MiB per file, 20 MiB total, `maxIssues` cap); one tool call at a time (`SERVER_BUSY`); an explicit result contract where only `status: complete` and `clean: true` means clean; stdout reserved for protocol, logs on stderr; scans run in `isolated` mode with the bundled rules.

Q40. Why are inline disable comments ignored in MCP? An agent that adds `// eslint-disable` would otherwise make its own scan pass. In MCP, findings are never hidden, and a covered finding includes `inlineDirective` info so the agent can see its comment didn't help.

H. Testing and error handling (existing implementation only)

Q41. How are rules tested? With `@typescript-eslint/rule-tester` through a shared helper (`tests/helpers/rule-tester.ts`), one test file per rule in `tests/rules/`, with `valid` and `invalid` arrays.

Invalid cases assert `messageId` and the exact autofix `output`. `CLAUDE.md` forbids re-declaring `RuleTester` in test files.

Q42. What else is tested? `tests/cli/` (commands, SARIF, baseline, MCP protocol, hooks, inline suppressions), `tests/ci/` (SARIF schema and persistence, PR diff logic, Action entrypoint security), `tests/integration/` (Express and Next.js sample scans), and `tests/integrity/` (temporary Git repositories). Many tests build throwaway Git repos in a temp directory (`tests/helpers/git-repo.ts`), so nothing touches the network.

Q43. How are errors handled so the tool never lies? Parser errors and foreign-plugin messages are separated from findings (`classifyMessage`). An explicitly requested ignored file exits 2 ("scan did not run") rather than reporting clean. MCP never labels partial scans clean. A missing/corrupt baseline is an error, not "no baseline". Integrity returns exit 2 if it cannot run.

Q44. How do you test the CLI as a real binary? Several tests spawn `dist/cli/index.js` with `spawnSync` (for example `tests/cli/standalone-cli.test.ts` , `tests/integrity/cli.test.ts`), so bundling issues are caught — that's how the `import.meta` shim problem was found.

I. Design decisions, trade-offs, limitations

Q45. Why AST heuristics instead of the TypeScript type checker? Speed and zero config. 17 of 18 rules need no type information, so no `tsconfig.json` . The docs benchmark (`docs/benchmarks.md` : 196 files, one Windows 11 / Node 20 machine, median of 3 runs) shows ~1.8 s vs ~8–12 s for type-aware ESLint. The cost: cross-file facts (is an imported function async?) aren't known unless the optional type-aware path is available.

Q46. What are the main limitations? Per-file analysis (no cross-file auth or taint tracking), name-based secret detection, heuristics for async detection, no semantic judgment of tests, and the fingerprint includes the line number. Docs list these under limitations (e.g., `docs/rules/require-auth-middleware.md` , `docs/integrity.md`).

Q47. How do you control false positives? Confidence tiers; `recommended` keeps noisy rules `off` / `warn` ; comment-aware rules (empty catch with a comment is allowed); context suppression (Electron/scripts for auth rules); placeholder/hash filters (secret rule); baseline mode for legacy code; and the philosophy "precision over recall".

Q48. Why is the Action a JavaScript action and not Docker? `runs: node20` starts immediately without pulling an image, and it can reuse the project's own installed binary when present.

J. Difficult follow-ups

Q49. "How is this different from Semgrep/CodeQL/SonarQube?" Repository evidence: `PROJECT-DEEP-DIVE.md` and `docs/benchmarks.md` position it as lightweight, AI-pattern-specific, ESLint-native, zero-config and GitHub-native, and `ROADMAP.md` explicitly says "We will NOT become a general-purpose security scanner." I haven't verified competitor internals from the repository, so I speak about *positioning*, not claims about their weaknesses.

Q50. "Can the autofix break code?" Yes, in principle. The secret fix needs the env var to exist; `void` in the floating-promise fix marks intent but doesn't handle rejection. That's why fixes are limited to four rules and others are only suggestions or no fix.

Q51. "Your fingerprint includes the line. Doesn't that defeat persistence?" Partly, yes: unchanged findings keep the same alert, but moved findings get a new fingerprint. The older guide's claim of line-shift stability is not accurate for the current code. A possible improvement is hashing a content-based anchor instead; I'd propose that rather than defend the claim.

Q52. "How would you add a new rule?" Per `CONTRIBUTING.md`: create the rule file in the right category folder with `createRule`; write `tests/rules/<name>.test.ts` with valid and invalid cases; **register it in 5 places** — `src/rules/index.ts`, `src/configs/recommended.ts`, `src/configs/strict.ts`, the preset rule maps in `cli/utils/eslint-runner.ts`, and `RULE_DOCS` in `cli/utils/sarif.ts`; write `docs/rules/<name>.md`; then run the full validation (`npm run typecheck && npm run test`).

Q53. "What would you do about a rule that needs cross-file info?" Document it as a limitation (as `require-auth-middleware` does) and consider the optional type-aware path or a project-level analysis; the roadmap lists cross-file analysis as future work.

---

7. Code walkthrough

For each file: purpose → key parts → inputs/outputs → connections → likely questions.

7.1 `src/index.ts`

- **Does:** defines the plugin object and exports named presets.
- **Key logic:** version lookup with ESM/CJS handling (`import.meta.url` vs `__dirname`) wrapped in `try/catch` with a fallback string.
- **Input/Output:** none at runtime; output is `{ meta, rules, configs }`.
- **Connects to:** `src/rules/index.ts` and `src/configs/*`; consumed by ESLint and by the runner's `loadPluginModuleFromCwd`.
- **Interviewer might ask:** *Why dual CJS/ESM?* — the plugin must load from either flat-config ESM or legacy CJS setups. *Why a fallback version?* — loading must never fail because `package.json` can't be found.

7.2 `src/rules/index.ts`

- **Does:** imports all 18 rules and builds `allRules` keyed by rule name (`'no-empty-catch': noEmptyCatch`); exports `RuleKey` type.
- **Ask:** *How are rules discovered?* — explicit imports, no dynamic file scanning; simple, type-safe, tree-shake friendly.

7.3 `src/rules/reliability/no-empty-catch.ts` (56 lines)

Walkthrough in 3.2 and 4.5. **Ask:** *What if the catch contains only a comment?* — not flagged. *What if `catch {}` has no parameter?* — still a `CatchClause`, `param` is `null`; the rule only looks at the body. *Why both `fix` and `suggest`?* — safe default insertion plus a user-visible option.

7.4 `src/rules/security/no-hardcoded-secret.ts`

Full walkthrough in 3.13. **Ask:** *Why three visitors?* — secrets appear as variables, member assignments and object properties. *Why* `value.length < 8` *?* — cuts short placeholders; trade-off is missing short real secrets.

7.5 `src/rules/security/no-eval-dynamic.ts`

Walkthrough in 4.5. **Ask:** *Why handle* `Function(...)` *without* `new` *?* — it behaves identically; a code comment in the file notes the earlier gap where only `NewExpression` was checked.

7.6 `src/rules/async/no-floating-promise.ts`

- **Does:** reports bare call statements that are promises and unhandled.
- **Pieces:** `nodeHasCatchClause` (does a local async function wrap its body in try/catch, without descending into nested functions), `isAsyncFunctionLike`, `isLocallyAsyncCallee`, `isKnownPromiseFactoryCall` (`fetch`, `Promise.resolve/reject/all/allSettled/race/any`), `isPromiseLikeByTypeInfo` (uses `esTreeNodeToTSNodeMap` + `getTypeChecker()` when available).
- **Output:** report with `floatingPromise` and a fix prefixing `void `.
- **Ask:** *What if the parser has no type info?* — `isPromiseLikeByTypeInfo` returns false; heuristics still work. *Why skip functions with internal try/catch?* — avoids flagging intentional fire-and-forget UI effects.

7.7 `src/rules/security/no-sql-string-concat.ts`

- **Pieces:** `variableMap` + `builderBindings` built in `VariableDeclarator` / `AssignmentExpression`; `isSqlSinkCall`; `isKnownQueryBuilderContext` (skip builders); `resolveExpression`; `isDynamicSqlExpression`.
- **Ask:** *How do you avoid flagging ORMs/query builders?* — builder-chain detection and an explicit allowance for awaited results.

7.8 `src/rules/security/require-auth-middleware.ts`

- **Pieces:** `PUBLIC_ROUTE_SEGMENTS`, `isPublicRoute` (first path segment after `/api`, `/api/vN`, `/vN`), `isRouteDefinition` / `isRouterLikeName`, `getBlanketAuth` (`use`, Fastify `addHook`), per-router `routerAuthScopes` map, `isAuthMiddleware` (identifiers, calls, members, arrays, Fastify option objects), `compileInternalPathPattern` (user regex with a clear error).
- **Options (schema):** `authMiddlewareNames`, `allowElectronApps`, `allowInternalScripts`, `internalPathPatterns`.
- **Ask:** *Why per-router tracking?* — an earlier bug leaked `router.use(auth)` state across routers (a regression test group "C3" exists).

7.9 `cli/index.ts`

- **Does:** builds the `commander` program, registers 12 commands, sets global error output, handles `unhandledRejection`.
- **Ask:** *Why separate* `register…Command` *functions?* — each command is self-contained and unit-testable.

7.10 `cli/utils/eslint-runner.ts`

Covered in 3.4. **Ask:** *Where does a raw ESLint message become an AI Guard finding?* — in `runEslint`'s result loop: `classifyMessage` → build `IssueDetail` with `ISSUE_CONFIDENCE` etc. → `applySuppressionPolicy`. *Why a module-level cache?* — avoid reloading ESLint/plugin/parser; invalidated on error.

7.11 `cli/commands/run.ts`

- **Does:** option parsing, preset resolution, calling `runEslint`, choosing output (SARIF, JSON, terminal), GitHub integration, exit code.
- **Exports used elsewhere:** `resolveExitCode`, `renderIssuesByFile`, `buildSignalSummary`, `renderSuppressedIssues`, `partitionByTier`.
- **Ask:** *Why print the banner conditionally?* — when stdout carries JSON/SARIF nothing else may be written there.

7.12 `cli/commands/changed.ts`

- **Does:** like `run` but file list from `getChangedFiles`; supports `--baseline` (via `filterBaselinedIssues`), `--integrity`, SARIF to file in CI, shared `handleResults`.
- **Ask:** *What happens with zero changed files?* — prints a warning (never silent) or emits JSON with zero counts; exit 0. With `--integrity` it still analyses config-only changes.

7.13 `cli/utils/git-diff.ts`

Covered in 3.6. **Ask:** *Why five strategies?* — shallow clones, detached HEAD and missing remotes are common in CI.

7.14 `cli/utils/sarif.ts`

Covered in 3.7. **Ask:** *Why dedupe tags?* — GitHub's schema validation rejects duplicate tags.

7.15 `cli/utils/baseline.ts`

- **Does:** `BaselineFile` type, `buildBaseline`, `loadBaseline` (returns `missing | invalid | ok`), `filterBaselinedIssues`, integrity acceptance helpers.
- **Key logic: count-based matching** — each baseline entry hides exactly one current finding; a second identical finding in the same file is reported as new even in `stable` mode.
- **Ask:** *Stable vs strict mode?* — `stable` = `file::rule::message` (survives line shifts); `strict` adds line and column.

7.16 `action/entrypoint.js`

Covered in 3.8. **Ask:** *Why parse SARIF for outputs?* — the CLI already wrote it; it's the single source of truth for counts. *Why `shell: false`?* — avoids injection through inputs.

7.17 `src/mcp/create-server.ts` and `src/mcp/utils/scan.ts`

Covered in 3.10. `scanFiles()` validates each path, extension, size and exclusion, then calls `runEslint(... isolated: true, inlineSuppressions: 'ignore')`, converts issues to `McpIssue` (`toMcpIssue`), sorts errors first and truncates with `RESULT_LIMIT`.

7.18 `cli/commands/claude-hook.ts`

Covered in 3.11. **Ask:** *How do you avoid blocking Claude forever?* — one block per unique finding set stored in a per-session state file; `stop_hook_active` handling; advisories never block.

7.19 `cli/utils/integrity/engine.ts`

- **Flow:** `analyzeIntegrity` → `resolveBase` → `listChanges` → classify files (JS vs config) → batch read blobs → parse (skipping unrelated sources via a text prefilter) → run analyzers → fingerprint, de-duplicate, remove accepted → sorted `IntegrityReport`.
- **Ask:** *How do you avoid reporting old problems?* — everything is a base-vs-head comparison; unchanged suppressions/skips cancel out (multiset comparison).

---

8. System design questions

8.1 "Design AI Guard." (HLD answer outline)

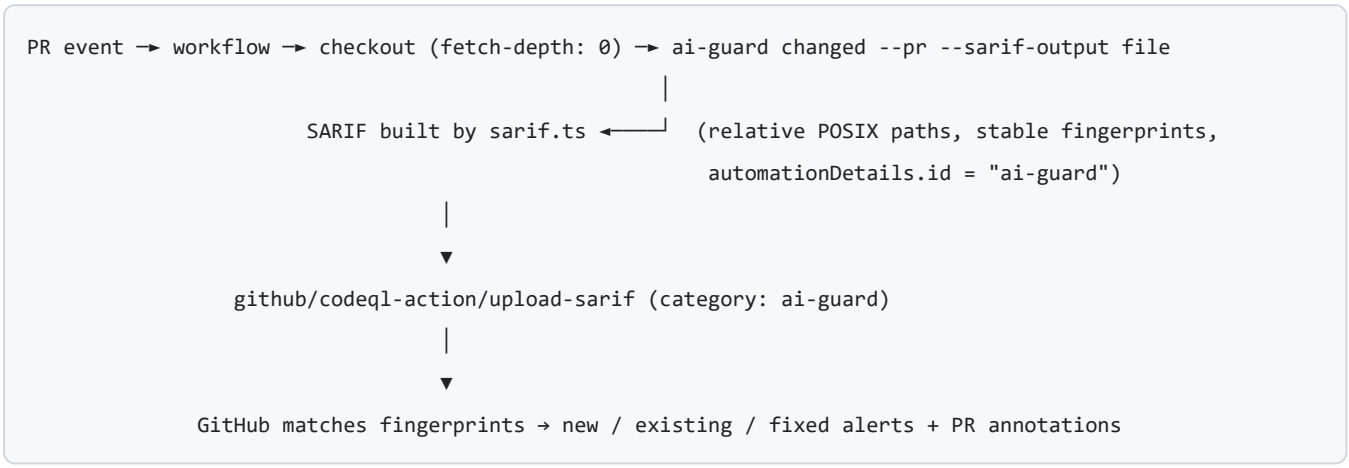
1. **Requirements.** Detect risky patterns in AI-generated JS/TS; fast; low false positives; works locally, in CI, and for agents.
2. **Core idea.** Reuse ESLint's parser/traversal; implement rules as visitors.
3. **Layers.** Rule layer (pure AST), Runner layer (config/parser/cache/normalise), Interface layer (CLI, Action, MCP, hooks), Reporting layer (terminal/JSON/SARIF/summary).
4. **Data contract.** `RunResult`.
5. **Scaling.** Incremental (`changed`) scans; explicit file lists; parser/ESLint caching; MCP/integrity limits.
6. **Failure handling.** Classification of parser/ecosystem errors; explicit exit codes; "never report clean when the scan didn't run".

```
interface layer      CLI · Action · MCP · Hooks
       |
orchestration        eslint-runner.ts  ── git-diff.ts / integrity engine
       |
analysis             ESLint + parser ─► rules (visitors)
       |
reporting            logger · sarif · github-summary · baseline · report
```

8.2 "How would it scale to a 5-million-line monorepo?"

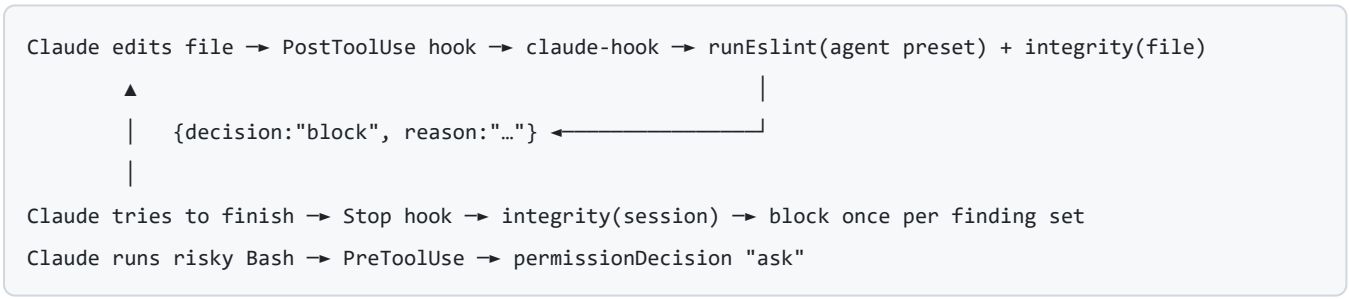
Based on the repository: use `changed` (scan only the diff), `--path` / `working-directory` scoping, `DEFAULT_IGNORE_PATTERNS`, cached ESLint instance. Sharding across workers or caching results per file hash is **not implemented — Not verified from repository**; present it as a design extension, clearly labelled as future work.

8.3 "Design the GitHub Code Scanning integration."



(`.github/workflows/ai-guard.yml` implements this flow for the repo itself.)

8.4 "Design the agent feedback loop."



State is kept in a small file under the OS temp directory (`ai-guard/claude-sessions`) keyed by session ID + repo.

8.5 LLD question: "Show me how you'd add a rule `no-console-log-secret`."

1. File `src/rules/security/no-console-log-secret.ts` using `createRule`.
2. Visitor `CallExpression`: callee `console.log` whose arguments contain an identifier matching `SECRET_NAME_PATTERN`.
3. `meta.messages`, `schema: []`, no autofix.
4. Register in the 5 places from `CONTRIBUTING.md` (`src/rules/index.ts`, `recommended.ts`, `strict.ts`, the runner's preset maps, `RULE_DOCS` in `sarif.ts`); the runner's metadata maps (`ISSUE_CONFIDENCE`, `ISSUE_CATEGORY`, `ISSUE_REMEDIATION`) and `docs/rules/…md` also need entries.
5. Tests: valid (logging a non-secret, `.length` of a secret) and invalid cases.

(This is a design exercise, not an existing rule.)

8.6 Data-flow summary for check integrity

```
Git base (merge-base / HEAD / session snapshot)          Working tree / index
        │ cat-file --batch (base text)                        │ read files
        └──────────────────┬──────────────────────────────────┘
                           ▼
        parse JS/TS (ESLint Linter + parser) → facts: directives, tests, assertions, as any
        parse configs (JSON/JSONC/YAML/AST) → effective values
                           ▼
        compare base vs head (multiset) → drafts → fingerprint → remove accepted
                           ▼
        IntegrityReport { findings[high|advisory], accepted, errors }
                           ▼
        terminal · JSON · SARIF · annotations · summary · hook JSON · MCP result
```

---

9. AI-assisted development questions

Facts the repository supports (use these; do not exaggerate):

- The project is *about* AI-assisted coding and ships `CLAUDE.md`, `init-context` (generates agent instruction files), a Claude Code plugin (`plugins/ai-guard/`) and hooks.
- Git history contains many commits by several author identities (`Yash Jadhav` , `YashJadhav21` , `Vaishnavi Sawant` / `Vaish16-code` , and one other). Some commit messages mention Claude (e.g. "Claude code plugin update"), and git history includes commits with a `Co-Authored-By` trailer (the 1.4.3 release commit has one).
- The 1.4.3 work (check integrity, hook protocol fix, inline-suppression policy, accuracy fixes) is documented in `CHANGELOG.md` and `docs/release/1.4.3-readiness-audit.md` , including the real-session verification that was done.
- Quality controls that exist in the repo: strict TypeScript, the shared rule tester, "valid and invalid" tests per rule, SARIF schema tests, a self-scan with the strict preset in CI, CI workflows, regression tests for found bugs.

Not verified from repository: which specific files you personally typed versus which an AI agent generated. Fill the blanks below with your own truth.

Q1. Did you use AI tools to build AI Guard? Suggested answer: "Yes — it's a tool about AI-assisted coding, and I used AI coding agents such as Claude Code during development, which is also visible in the repository (the Claude plugin, `CLAUDE.md` , and co-author trailers in git history). I treated the agent like a fast junior pair-programmer: I defined the problem and the design, it produced drafts, and I reviewed everything against tests and real behaviour before keeping it."

Q2. Then what exactly did you contribute? *(Fill in honestly — examples of things you can discuss only if true.)*

- "[I chose the problem and the scope: deterministic AST rules, no LLM, zero-config.]"
- "[I decided the rule list, severities and confidence tiers.]"

- "[I designed or reviewed the architecture: runner as the single core, `RunResult` contract, SARIF constraints.]"
- "[I wrote / debugged rule X myself.]" — name one rule you can explain line by line (the secret rule in 3.13 is a good choice).
- "[I verified behaviour end to end: ran the CLI on real projects, checked GitHub Code Scanning output, etc.]"

Q3. How did you review AI-generated code? "I used the repo's own safety net: every change must pass `npm run typecheck`, the Vitest suite (rule `valid` / `invalid` tests, CLI and integration tests), the strict self-scan of `src` and `cli`, and ESLint. For behaviour that unit tests can't prove — like Claude hooks — I asked for real-session tests, and the audit document records that, plus the bugs they found."

Q4. Give an example where AI-generated code was wrong and you caught it. *(Repository-backed examples.)*

- The old interview guide claimed the secret rule used entropy and that SARIF fingerprints survive line shifts; reading the code showed neither is true (Section 0).
- Plain-stdout hook output looked fine in a protocol test but is not delivered to Claude by Claude Code's hook protocol; the fix and its docs are in the changelog.
- A CI-step "removed" false positive and a duplicate-test false positive were found by replaying real open-source commits (recorded in the readiness audit) and fixed with regression tests.
- Inline `eslint-disable` comments could silently hide findings; fixed with an explicit policy.

Q5. How do you ensure code quality when AI writes much of the code? "Tests first as a gate, not an afterthought: negative and positive cases per rule, regression tests for each bug, a self-scan so the tool must pass its own rules, strict TypeScript, and documentation of limitations. I also don't trust numbers in docs — I check them against the code."

Q6. Isn't it ironic to build an AI-code linter with AI? "It's actually the point: AI makes people faster, so verification must be automated and deterministic. Using AI to build the tool lets me experience the same failure modes the rules target — and the repository shows we caught them with tests."

Q7. Would you claim you wrote every line? "No. I own the design decisions, the review and the verification; an AI agent helped with implementation. I can explain any file I present to you line by line — pick one." *(Only say this if you have truly studied the file; use the rule walkthroughs in this guide.)*

Q8. What would you do differently? Suggested, repository-grounded: keep a single source of truth for preset tables (currently both `src/configs/*` and the runner have preset maps); keep documentation numbers generated from code (several stale counts exist); add value-based secret detection.

---

10. Final rapid revision

10.1 Two-minute introduction

> "AI Guard is an open-source ESLint plugin and toolchain I built to catch the bugs AI coding assistants commonly introduce in JavaScript and TypeScript — things like unawaited promises, empty catch blocks,

> hardcoded secrets, SQL string building and missing auth on routes. It has 18 deterministic rules in four categories and four presets, and it's fully static: it parses code into an AST and runs visitor-based rules, with no LLM involved, so results are fast and repeatable. It runs in four places: as a zero-config CLI, as a GitHub Action that uploads SARIF so findings show up in GitHub Code Scanning, as an MCP server that AI agents can call, and as Claude Code hooks that feed findings back to the agent right after it edits a file. The core is one ESLint runner that every interface reuses, and I focused on trust: confidence tiers, baselines for legacy code, a policy so `eslint-disable` comments can't silently hide findings, and a new 'check integrity' feature that flags when a change weakens tests or CI. It's tested with over a thousand Vitest tests, and the project scans its own source in CI."

10.2 Five-minute architecture explanation

1. **Problem (30 s).** AI code looks clean but hides async, error-handling and security defects; normal linters either miss them or need slow type-checking.
2. **Plugin layer (60 s).** `src/rules/` holds 18 rules created with `ESLintUtils.RuleCreator`. Each returns visitors keyed by AST node type (`CatchClause`, `CallExpression`, `VariableDeclarator`…). Rules use `context.sourceCode` for comments and scope, and `context.report` with optional `fix` / `suggest`. `src/index.ts` exports the rules and four presets.
3. **Runner (60 s).** `cli/utils/eslint-runner.ts` builds an in-memory flat config (`overrideConfigFile: true`), loads the plugin and parser in parallel, caches the ESLint instance, lints, and converts raw messages to `IssueDetail`s with confidence, category and remediation. It separates parser errors and foreign-plugin messages from real findings and applies the inline-suppression policy.
4. **Interfaces (60 s).** CLI commands (`run`, `changed`, `baseline`, `report`, `integrity`…) are registered with `commander`. `changed` uses Git merge-base strategies to scan only a PR's files. The Action spawns the CLI with an argument array and no shell, then derives outputs from the SARIF file. The MCP server exposes five read-only tools with zod schemas and strict limits. The Claude hook speaks the documented JSON hook protocol.
5. **Reporting (45 s).** SARIF 2.1.0 with repo-relative paths, fixed `automationDetails.id`, and SHA-256 fingerprints; step summary and annotations; JSON; HTML report; exit codes via `--fail-on`.
6. **Quality and trade-offs (45 s).** Vitest with valid/invalid cases per rule, CLI/CI/integration tests, real Git temp-repo tests, strict self-scan. Trade-offs: heuristics instead of full type info (fast, but per-file), name-based secret detection, advisory vs blocking tiers to protect trust.

10.3 The 20 most important questions (short answers)

1. **What is AI Guard?** Deterministic AST-based ESLint plugin + CLI + Action + MCP + hooks for AI-generated JS/TS issues.
2. **Does it use AI?** No; it is static analysis, repeatable and free of API cost.
3. **Why AST not regex?** Regex can't separate code from strings/comments, can't see structure or scope; AST nodes are exact and fixes are range-safe.
4. **How does a rule work?** `create(context)` returns `{ NodeType(node) {…} }`; ESLint walks the AST once and calls handlers; `context.report` emits findings.
5. **How is zero-config achieved?** `new ESLint({ overrideConfigFile: true, overrideConfig })` with an in-memory flat config.
6. **How many rules/presets?** 18 rules; 4 presets (`recommended`, `strict`, `security`, `agent`).

7. **How does `no-empty-catch` work?** `CatchClause` with empty body and no comments → report + fix/suggestion.
8. **How does `no-hardcoded-secret` work?** Secret-like name + plain string value, not a placeholder, length ≥ 8, not a hash name → report; fix to `process.env.NAME`.
9. **How does `no-floating-promise` work?** Bare call statement; callee is local async function, known promise factory, or type says Promise; skip handled/`void`.
10. **How does `changed` work?** Merge-base with PR base → changed files → scan only those; five fallback strategies.
11. **What is SARIF and why?** JSON standard for static-analysis results; GitHub Code Scanning shows persistent alerts from it.
12. **How are SARIF alerts kept stable?** Constant `automationDetails.id`, `partialFingerprints["ai-guard/v1"]`, fixed upload category.
13. **How does the Action run the tool?** Node20 JS action; builds argv array; `spawnSync` with `shell:false`; reads SARIF for outputs.
14. **What is MCP?** Protocol for AI clients to call tools; here 5 read-only scanning tools over STDIO.
15. **How do Claude hooks work?** Stdin JSON event → scan/integrity → one JSON object on stdout, exit 0 (`decision:"block"`, `reason`, `additionalContext`).
16. **How do you stop endless blocking?** One block per unique finding set per session; advisories never block.
17. **What is baseline?** Snapshot of existing findings; only new ones reported; count-based matching.
18. **What is check integrity?** Compares base vs working tree to find weakened checks (new suppressions, skipped/removed tests, lowered configs, removed CI steps); high vs advisory tiers.
19. **How is it tested?** Vitest + RuleTester valid/invalid cases; CLI, SARIF schema, Git, MCP, hook and integrity tests; temp Git repos; self-scan.
20. **Main limitations?** Per-file heuristics, name-based secrets, no cross-file analysis, fingerprint includes line, heuristic async detection without types.

10.4 Important definitions

- **AST:** tree of syntax nodes for source code.
- **ESTree:** the standard AST format used by ESLint/espree.
- **Visitor:** object mapping node types to handlers, invoked during traversal.
- **Scope:** region where a variable name is valid; used to resolve identifiers.
- **Static analysis:** analysing code without executing it.
- **Deterministic:** same input always produces the same output.
- **False positive:** valid code flagged; **false negative:** bad code missed.
- **Autofix:** automatic safe edit; **suggestion:** optional edit chosen by the user.
- **Flat config:** ESLint's array-based configuration.
- **SARIF:** Static Analysis Results Interchange Format.
- **Fingerprint:** stable hash identifying a finding across runs.
- **Merge-base:** the most recent common ancestor commit of two branches.
- **MCP:** Model Context Protocol for AI tool access.
- **Hook:** command invoked by Claude Code on a lifecycle event.
- **Baseline:** accepted list of existing findings.
- **Confidence tier:** `high/medium/low/informational` reliability label.

- **Check integrity:** detecting changes that weaken safety checks.
- **Precision vs recall:** fraction of reports that are correct vs fraction of real issues found.

10.5 Last-minute checklist

- [ ] Say **1.4.3 (unreleased), last release 1.4.2**; **18 rules, 4 presets, 12 CLI commands, 5 MCP tools**; tests: **"over a thousand (1,042 recorded in the release audit)"**.
- [ ] Can you draw the HLD diagram (interfaces → runner → parser/rules → reporters) from memory?
- [ ] Can you walk through `no-hardcoded-secret` (3 visitors, 5 filters, fix) and `no-empty-catch` line by line?
- [ ] Can you draw the AST for `try { run(); } catch (e) {}`?
- [ ] Know what **not** to claim: no ML/LLM in the engine; not an AI detector; not all rules autofix (4 do); no entropy detection; fingerprint includes the line; don't claim cross-file analysis.
- [ ] Know the **limitations** and one honest improvement (value-based secret detection, shared preset source of truth).
- [ ] Prepare your **personal contribution** statement (Section 9) — be specific and truthful.
- [ ] Have one **bug story**: e.g. hook output not reaching Claude, SARIF/Git edge cases, or the router auth-state leak.
- [ ] If asked about something not in this guide: say what the code does, then say you'd verify before claiming more.
- [ ] Don't mention other projects; keep everything about AI Guard.